Polina Khlystun

DATA SCIENCE CAMP
2026

Technical Task Solution

TASK 1 2

TASK 2 5

TASK 3 11

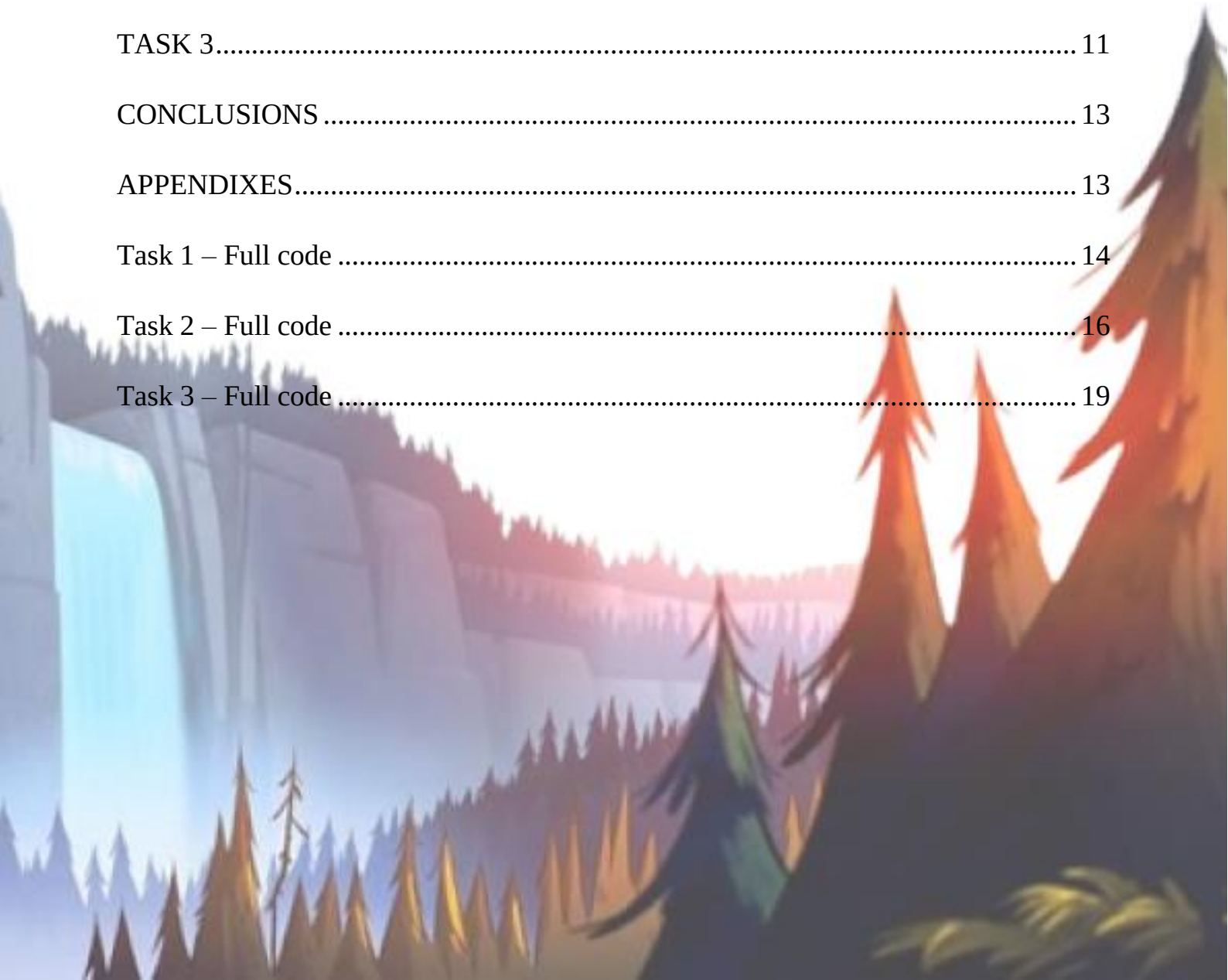
CONCLUSIONS 13

APPENDIXES 13

Task 1 – Full code 14

Task 2 – Full code 16

Task 3 – Full code 19



TASK 1

To complete the task, the **matplotlib** library for the Python programming language was used. The library was installed using the command:

```
pip install matplotlib
```

in the Python environment.

A sequence of 26 random integer values in the range from 0 to 26 was generated. These values represent the heights of a mountain terrain (see Listing 1.1).

Listing 1.1 – Random integer values

```
values = [random.randint(0, 26) for _ in range(26)]
```

To determine the deepest lakes, the function **`deepest_lake_depth`** was implemented. This function iterates over all possible pairs of terrain points, treating them as potential lake boundaries (see Listing 1.2). For each pair, it checks whether all values between the boundaries are lower than the water level, which allows detecting a closed basin.

Listing 1.2 – Function to determine the deepest lakes

```
def deepest_lake_depth(heights):
  n = len(heights)
  max_depth = 0
  lakes = []

  # iterate over all possible pairs of boundaries
  for i in range(n):
    for j in range(i + 2, n):
      water_level = min(heights[i], heights[j])
      inside = heights[i+1:j]

      # skip if there is no space between
      if not inside: continue

      bottom = min(inside)

      # check if a valid lake is formed
      if all(h < water_level for h in inside):
```

```
        depth = water_level - bottom

        # new max depth found
        if depth > max_depth:
          max_depth = depth
          lakes = [(i, j, depth, water_level, bottom)]

        # another lake with the same max depth
        elif depth == max_depth:

          # avoid nested duplicate
          if not any(i >= l[0] and j <= l[1] for l in lakes):
            lakes.append((i, j, depth, water_level, bottom))

    return max_depth, lakes
```

The lake depth is calculated as the difference between the water level and the lowest point inside the basin. The function stores all lakes with the maximum depth and returns them for further processing.

The algorithm's results are visualized using a graph: the mountain terrain is displayed as a continuous line, each deepest lake is highlighted with a distinct color and style, and the water level is shown using dashed lines.

Figure 1.1 illustrates the algorithm's operation.

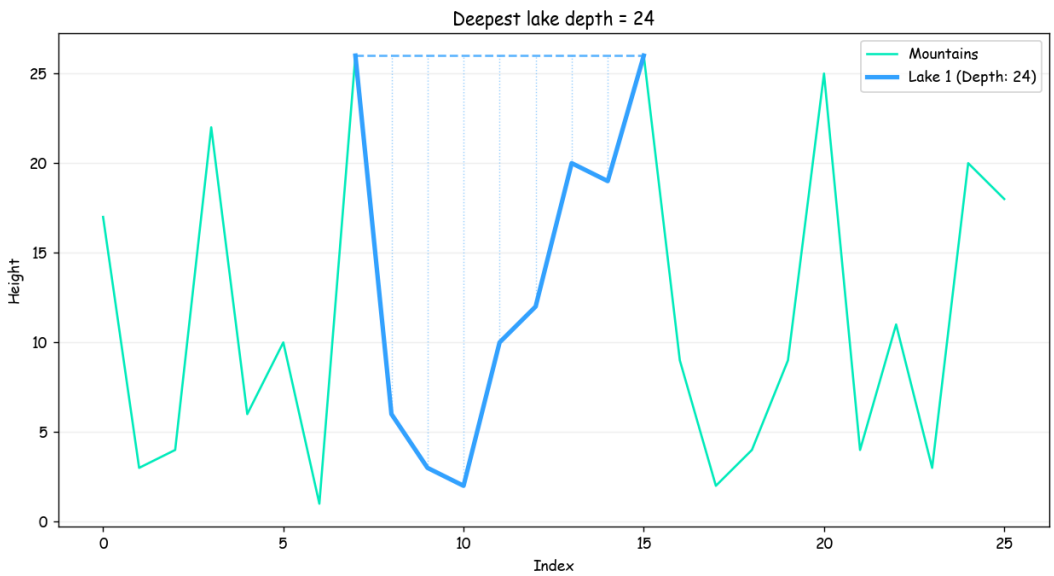


Figure 1.1 – Program execution result

The probability of generating two lakes with the same maximum depth is approximately 1-2% under uniform random distribution. An example of such a case is shown in Figure 1.2.

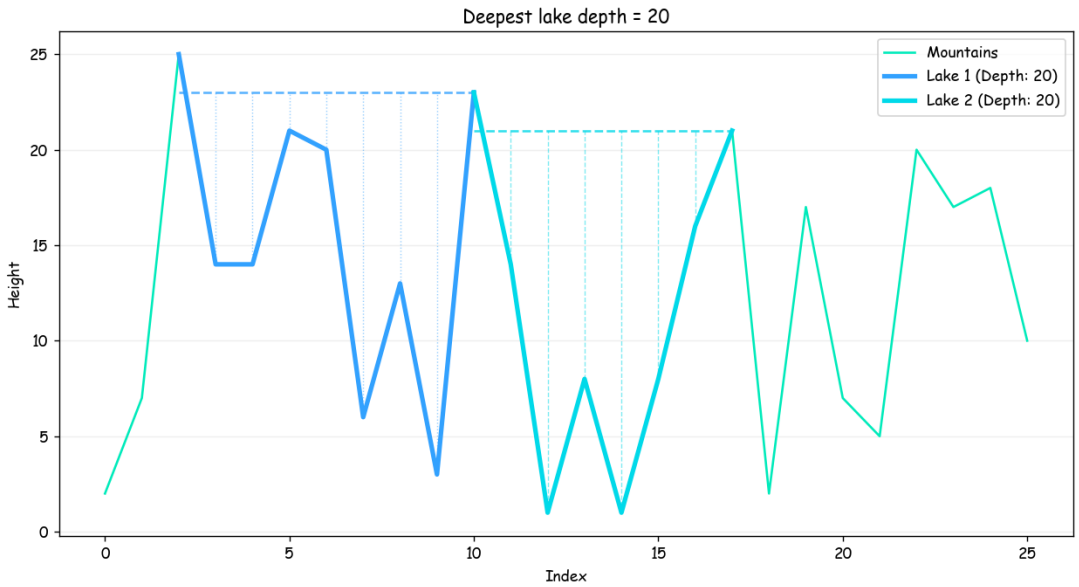


Figure 1.2 – Two lakes with the same maximum depth

The probability of generating three lakes with the same maximum depth is less than 1% and occurs very rarely. Nevertheless, this case was accounted for in the program (see Figure 1.3).

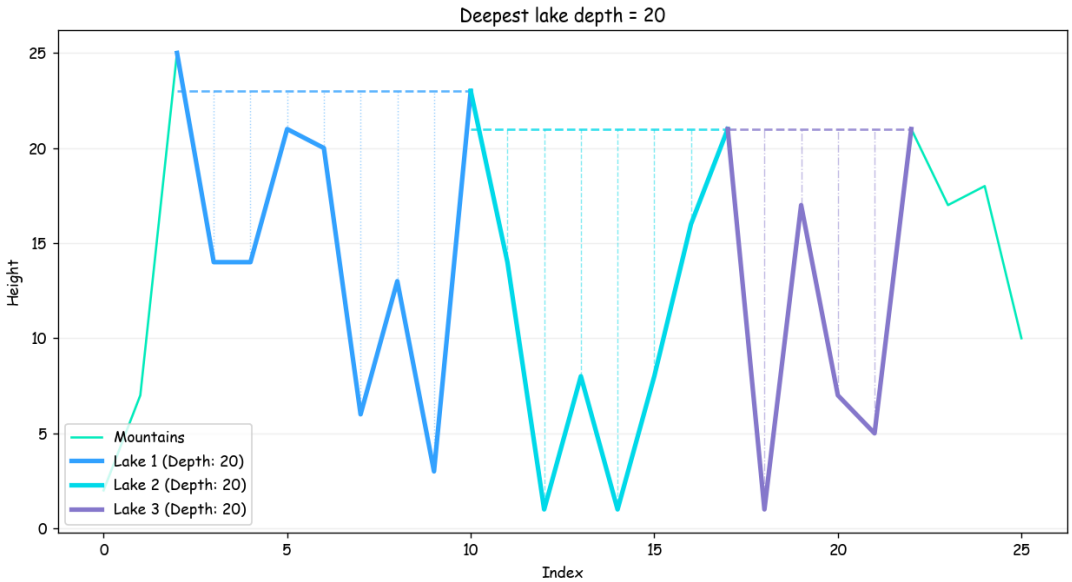


Figure 1.3 – Three lakes with the same maximum depth

The full source code of the program is provided in the **Appendix "Task 1 – Full code"**.

TASK 2

To generate and visualize the matrix, the **NumPy** and **Matplotlib** libraries were used (see Listing 2.1).

Listing 2.1 – Importing required libraries

```
import numpy as np
import matplotlib.pyplot as plt
```

A binary matrix of size **26x26** was generated as the initial state of Conway's *Game of Life*. The code is shown in Listing 2.2, and the result of execution is present in Figure 2.1.

Listing 2.2. – Grid generation

```
grid = np.random.randint(0, 2, (26, 26))
```

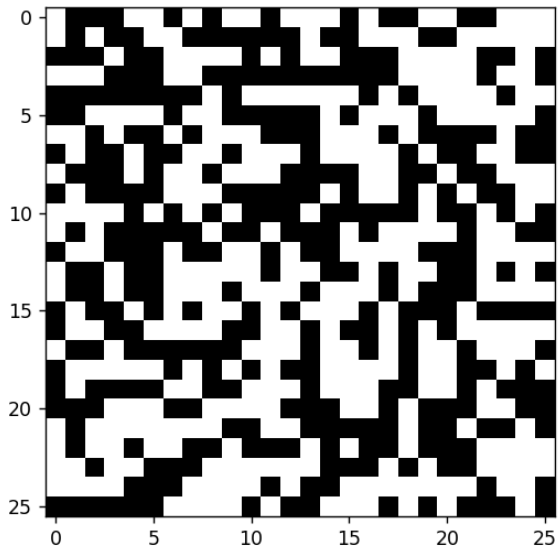


Figure 2.1 – Initial random grid ("digital noise")

At the next stage, a function was implemented that allows each cell to "observe" its surroundings. The function **`count_neighbors`** checks all eight neighboring cells, ignores the cell itself, and ensures that the grid boundaries are not exceeded.

The implementation is shown in Listing 2.3, and the conceptual behavior is illustrated in Figure 2.2.

Listing 2.3 – Neighbor counting algorithm

```
def count_neighbors(grid, row, col):
  neighbors = 0

  for i in range(row - 1, row + 2):
    for j in range(col - 1, col + 2):

      # Check grid boundaries
      if i < 0 or j < 0 or i >= grid.shape[0] or j >= grid.shape[1]:
        continue

      # Skip the cell itself
      if i == row and j == col:
        continue

      neighbors += grid[i][j]

  return neighbors
```

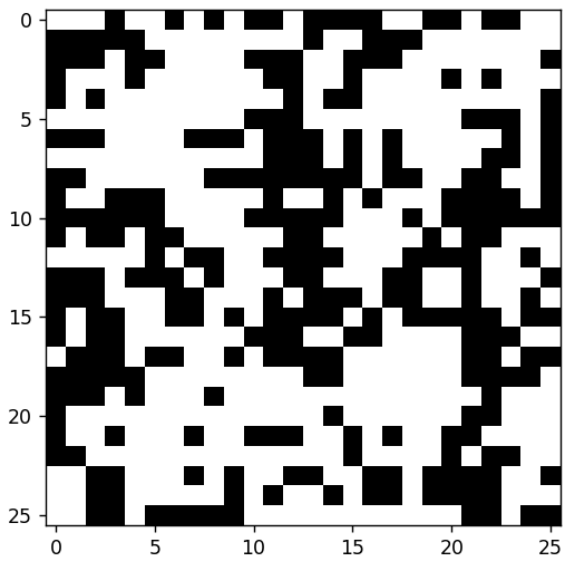


Figure 2.2 – Neighbor evaluation mechanism

After implementing neighbor detection, the classical rules of Conway's Game of Life were introduced. From this point, cells no longer exist randomly but interact with each other according to strict rules. The function **`next_cell_state`** implements the following rules:

- a living cell with two or three neighbors remains alive;
- a living cell with fewer than two or more than three neighbors dies;
- a dead cell with exactly three neighbors becomes alive.

The implementation is presented in Listing 2.4, and the system behavior is shown in Figure 2.3.

Listing 2.4 – Game of Life rules

```
def next_cell_state(grid, row, col):
  alive = grid[row][col]
  neighbors = count_neighbors(grid, row, col)

  if alive == 1:
    if neighbors == 2 or neighbors == 3:
      return 1
    else:
      return 0
  else:
    if neighbors == 3:
      return 1
    else:
      return 0
```

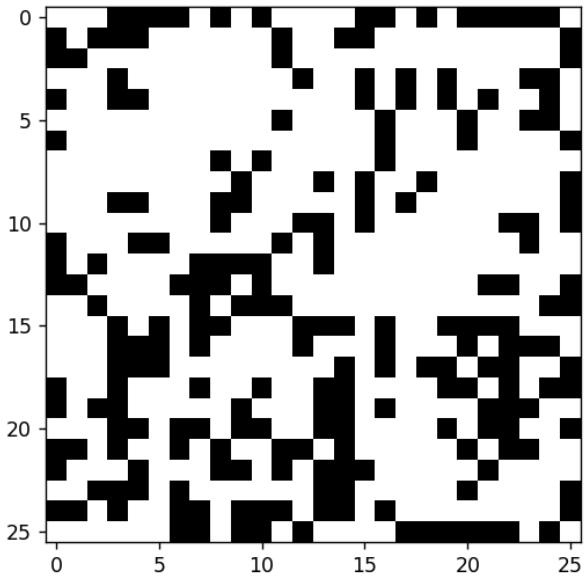


Figure 2.3 – System stabilization example

To update the entire grid simultaneously, the function **`next_step(grid)`** was implemented.

This function applies the rules to **all cells at once**, preventing intermediate updates from influencing the current generation.

The code is shown in Listing 2.5, and the result is illustrated in Figure 2.4.

Listing 2.5 – Evolution engine

```
01 def next_step(grid):
02   new_grid = np.zeros_like(grid)
03
04   for i in range(grid.shape[0]):
05     for j in range(grid.shape[1]):
06       new_grid[i][j] = next_cell_state(grid, i, j)
07
08   return new_grid
```

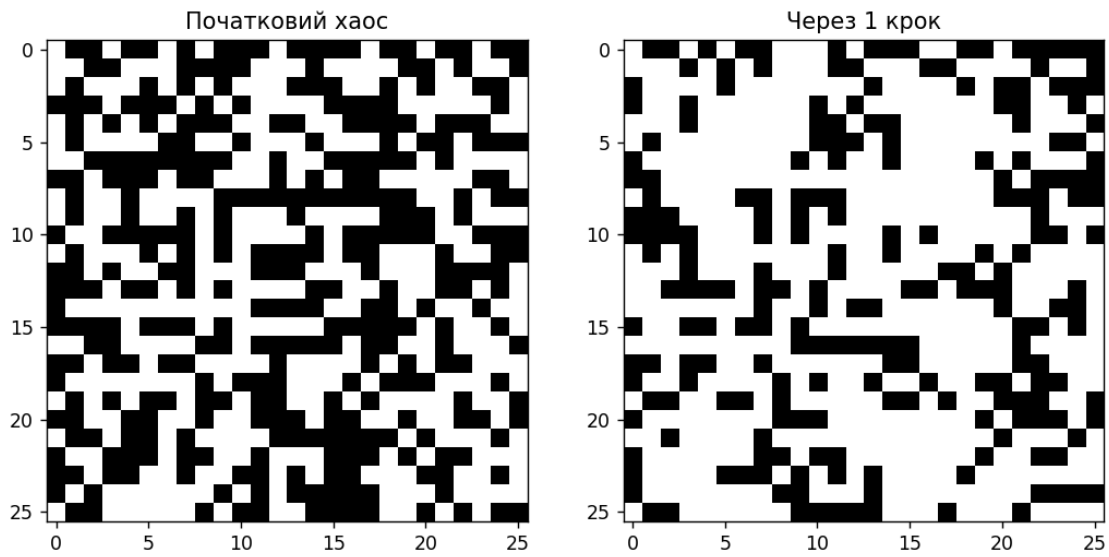


Figure 2.4 – Initial chaos and the first evolution step

The **Tkinter** library was added to display a dialog window requesting the number of evolution steps from the user. If the input is invalid or exceeds the allowed limit, a default value is applied.

The corresponding code is shown in Listing 2.6, and the dialog window is presented in Figure 2.5.

Listing 2.6 – User input for the number of evolution steps

```
01 import tkinter as tk
02 from tkinter import simpledialog
03
04 root = tk.Tk()
05 root.withdraw() # Hide main Tk window
06
07 user_input = simpledialog.askstring("Input", "How many evolution
   steps to simulate? (max 50):")
08
```

```
try:
  user_steps = int(user_input) if user_input else 20
except ValueError:
  user_steps = 20

if user_steps > 50:
  print("Too many steps requested. Limiting to 20 steps")
  user_steps = 20
```

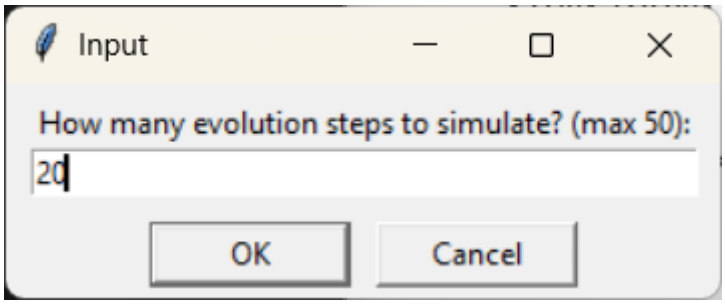


Figure 2.5 – Dialog window for selecting the number of steps

To improve visualization, an **animation** was added along with a color scheme that reflects the **age of each cell**. A separate matrix **`age_grid`** tracks how long each cell remains alive.

A custom colormap is used:

- light blue – newly born cells;
- pink – cells living more than five generations.

The corresponding code is shown in Listing 2.7, and the visualization result is present in Figure 2.6.

Listing 2.7 – Visualization and cell age coloring

```
# Cell age grid (used for coloring)
age_grid = np.zeros_like(grid, dtype=float)

# Custom colormap (blue → pink)
colors = ["#dcf2ff","#32a1fe","#436b9b","#f4709c","#b83969"]
custom_cmap = LinearSegmentedColormap.from_list("blue_pink_life",
colors)

# Visualization loop
plt.ion()
fig, ax = plt.subplots(figsize=(8, 8))

for i in range(user_steps):
```

```
    # Next generation
    grid = next_step(grid)

    # Update age grid
    age_grid[grid == 1] += 1
    age_grid[grid == 0] = 0

    ax.clear()

    img = ax.imshow(age_grid, cmap=custom_cmap, interpolation='nearest')

    ax.set_title(f"Generation #{i + 1}")
    ax.axis("off")

    # Footnote
    footnote = "LEGEND:\nLight blue — newborn cell\nPink — living more than 5 generations"
    ax.text(0, 28, footnote, fontsize=10, color="gray", style="italic")

    # Speed
    plt.pause(0.15)
```

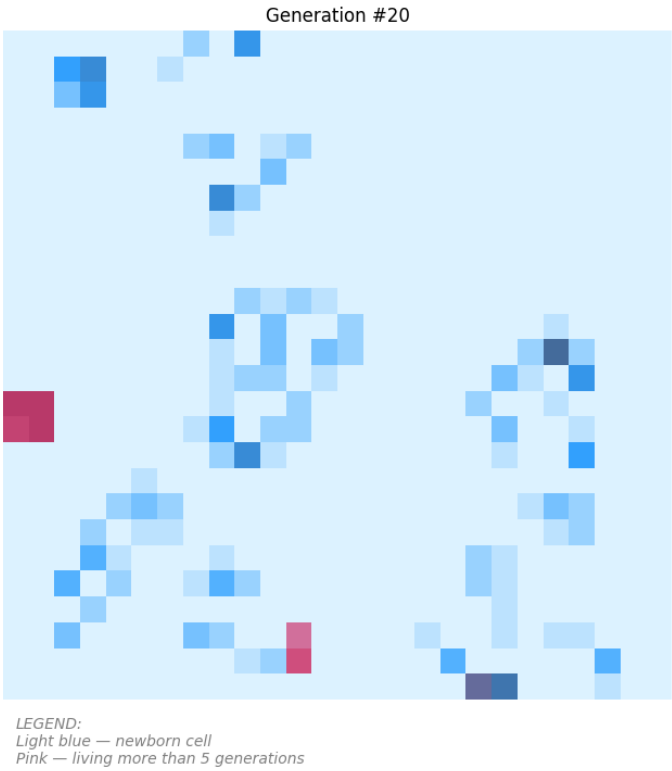


Figure 2.6 – Final application window

The full source code of the program is provided in the **Appendix "Task 2 – Full code"**.

TASK 3

A simple program was implemented to calculate the probability of obtaining “H” in the next coin flip based on previous observations (see Listing 3.1).

Listing 3.1 – Bayesian probability estimation for coin flips

```
import numpy as np

# Probabilities of getting Head for each coin
p = np.array([0.12, 0.27, 0.21, 0.96])

# Initial prior probabilities (all coins are equally likely)
posterior = np.ones(4) / 4

# Observed sequence of flips
observations = ['H', 'H', 'H', 'T', 'H', 'T', 'H', 'H', 'H']

results = []

# Bayesian updating
for o in observations:
    likelihood = p if o == 'H' else (1 - p)
    posterior = posterior * likelihood
    posterior /= posterior.sum()

    # Probability of Head in the next flip
    results.append(round((posterior * p).sum(), 2))

print(results)
```

After each flip, the probabilities of the possible coins are updated using Bayes’ rule, and the probability of “H” in the next flip is recalculated.

A graphical user interface was additionally implemented using the **Tkinter** library to visualize the results and allow user interaction (Figure 3.1).

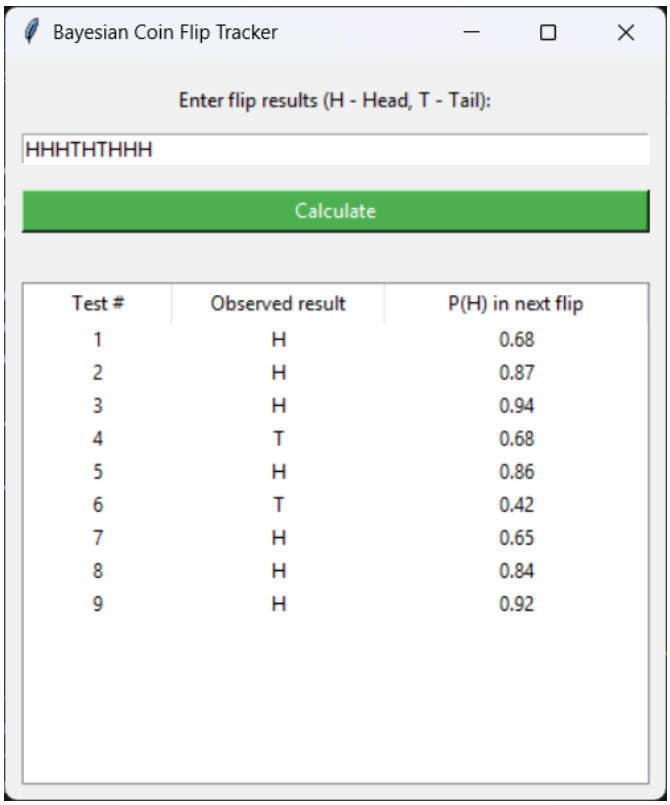
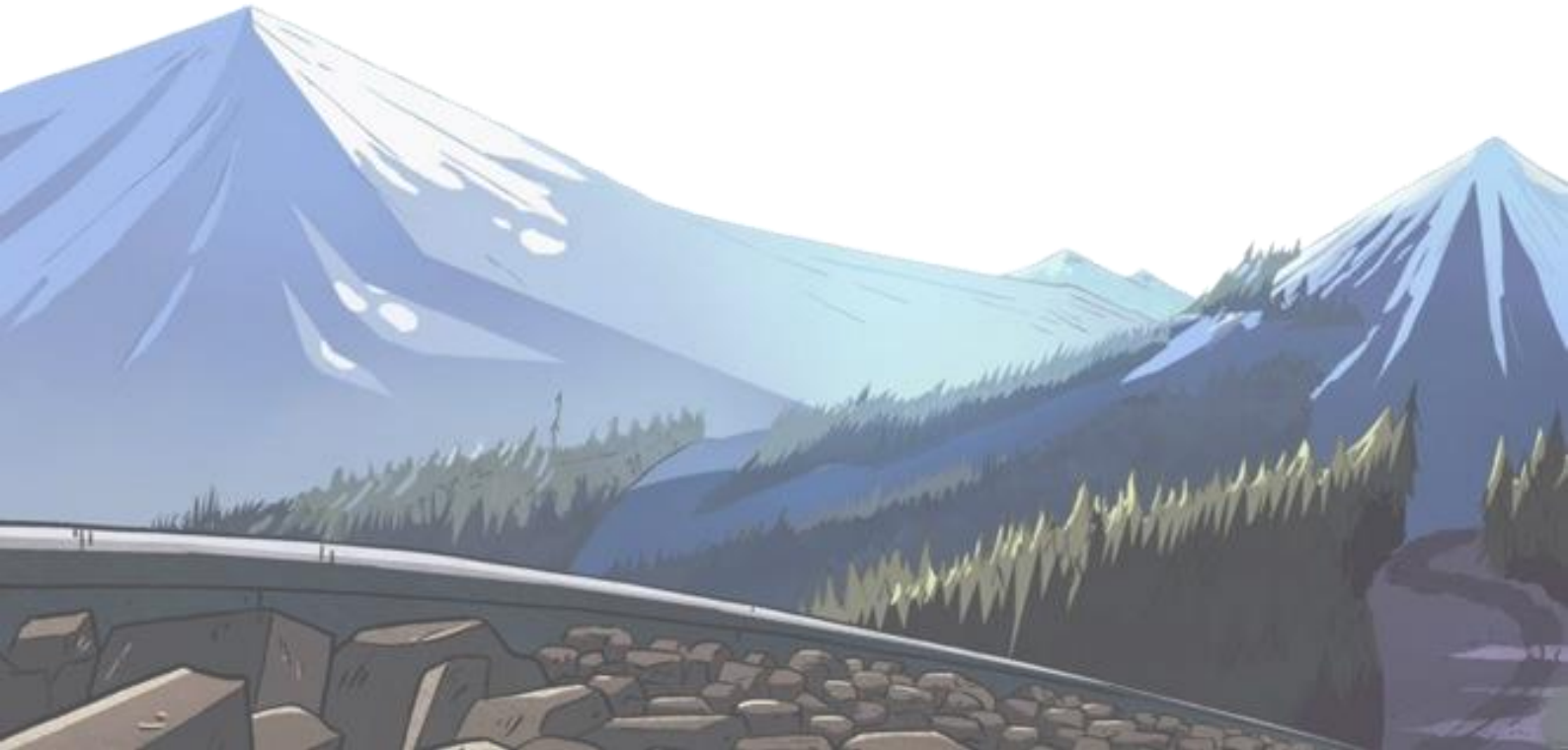


Figure 3.1 – Main application window

The full source code of the program is provided in the **Appendix "Task 3 – Full code"**.



CONCLUSIONS

Working on these tasks was interesting and informative. Each of them provided an opportunity to apply Python in different ways to solve practical problems – from data processing and algorithms to visualization and interactivity.I was able to implement the algorithms correctly, account for special cases, and add clear and convenient visualizations, making the programs more understandable and intuitive.

These exercises also helped me strengthen my skills in programming and working with libraries such as NumPy, Matplotlib, and Tkinter, as well as understand how to combine algorithms with a user interface for better data presentation. Overall, working on these test tasks was both engaging and useful, and I feel confident that I successfully completed all of them.

APPENDIXES

Task 1 – Full code

```python
import random
import matplotlib.pyplot as plt

# set custom font for vizual
plt.rcParams["font.family"] = "Comic Sans MS"

# generate 26 random integer values
values = [random.randint(0, 26) for _ in range(26)]

# values = [
#     2, 7, 25, 14, 14, 21, 20, 6, 13, 3,
#     23, 14, 1, 8, 1, 8, 16, 21, 1, 17,
#     7, 5, 21, 17, 18, 10
# ]

# finds the deepest lake/lakes in the given height
def deepest_lake_depth(heights):
  n = len(heights)
  max_depth = 0
  lakes = []

  # iterate over all possible pairs of boundaries
  for i in range(n):
    for j in range(i + 2, n):
      water_level = min(heights[i], heights[j])
      inside = heights[i+1:j]

      # skip if there is no space between
      if not inside: continue

      bottom = min(inside)

      # check if a valid lake is formed
      if all(h < water_level for h in inside):
        depth = water_level - bottom

        # new max depth found
        if depth > max_depth:
          max_depth = depth
          lakes = [(i, j, depth, water_level, bottom)]

        # another lake with the same max depth
        elif depth == max_depth:

          # avoid nested duplicate
          if not any(i >= l[0] and j <= l[1] for l in lakes):
            lakes.append((i, j, depth, water_level, bottom))

```

```python
  return max_depth, lakes

# run lake algorithm
max_depth, found_lakes = deepest_lake_depth(values)

# plot setup
plt.figure(figsize=(12, 6))
plt.plot(values, color="#00ebbb", linewidth=1.5, label="Mountains")

# color and style for lakes
colors = ["#32a1fe", "#00d9e9", "#8577cb"]
styles = [":", "--", '-.']

# vizual each detected lake
for idx, (i, j, depth, water_level, bottom) in enumerate(found_lakes):
  color = colors[idx % len(colors)]
  style = styles[idx % len(styles)]

  # lake boundaries
  plt.plot(range(i, j+1), values[i:j+1], color=color, linewidth=3, label=f"Lake {idx+1} (Depth: {depth})")

  # water level
  plt.hlines(y=water_level, xmin=i, xmax=j, colors=color, linestyles="--", alpha=0.8)

  # vertical water filling
  for k in range(i+1, j):
    plt.vlines(x=k, ymin=values[k], ymax=water_level, color=color, linestyles=style, linewidth=0.8, alpha=0.5)

# final plot styling
plt.title(f"Deepest lake depth = {depth}")
plt.xlabel("Index")
plt.ylabel("Height")
plt.legend()
plt.grid(axis='y', alpha=0.2)
plt.show()
```

Task 2 – Full code

```python
import numpy as np
import matplotlib.pyplot as plt
from matplotlib.colors import LinearSegmentedColormap

import tkinter as tk
from tkinter import simpledialog

# Generate initial grid (26x26) with random 0/1 values
grid = np.random.randint(0, 2, (26, 26))

# Count alive neighbors for a single cell
def count_neighbors(grid, row, col):
  neighbors = 0

  for i in range(row - 1, row + 2):
    for j in range(col - 1, col + 2):

      # Check grid boundaries
      if i < 0 or j < 0 or i >= grid.shape[0] or j >= grid.shape[1]:
        continue

      # Skip the cell itself
      if i == row and j == col:
        continue

      neighbors += grid[i][j]

  return neighbors

# Determine next state of a single cell
def next_cell_state(grid, row, col):
  alive = grid[row][col]
  neighbors = count_neighbors(grid, row, col)

  if alive == 1:
    if neighbors == 2 or neighbors == 3:
      return 1
    else:
      return 0
  else:
    if neighbors == 3:
      return 1
    else:
      return 0

# Perform one full evolution step
def next_step(grid):
```

```python
  new_grid = np.zeros_like(grid)

  for i in range(grid.shape[0]):
    for j in range(grid.shape[1]):
      new_grid[i][j] = next_cell_state(grid, i, j)

  return new_grid

# Ask user for number of steps
root = tk.Tk()
root.withdraw() # Hide main Tk window

user_input = simpledialog.askstring("Input", "How many evolution steps to simulate? (max 50):")

try:
  user_steps = int(user_input) if user_input else 20
except ValueError:
  user_steps = 20

if user_steps > 50:
  print("Too many steps requested. Limiting to 20 steps")
  user_steps = 20

# Cell age grid (used for coloring)
age_grid = np.zeros_like(grid, dtype=float)

# Custom colormap (blue → pink)
colors = ["#dcf2ff","#32a1fe","#436b9b","#f4709c","#b83969"]
custom_cmap = LinearSegmentedColormap.from_list("blue_pink_life", colors)

# Visualization loop
plt.ion()
fig, ax = plt.subplots(figsize=(8, 8))

for i in range(user_steps):
  # Next generation
  grid = next_step(grid)

  # Update age grid
  age_grid[grid == 1] += 1
  age_grid[grid == 0] = 0

  ax.clear()

  img = ax.imshow(age_grid, cmap=custom_cmap, interpolation='nearest')

  ax.set_title(f"Generation #{i + 1}")
  ax.axis("off")
```

```python

  # Footnote
  footnote = "LEGEND:\nLight blue – newborn cell\nPink – living more than 5 generations"
  ax.text(0, 28, footnote, fontsize=10, color="gray", style="italic")

  # Speed
  plt.pause(0.15)

# Finalize visualization
plt.ioff()
plt.show()
```

Task 3 – Full code

```python
import tkinter as tk
from tkinter import ttk, messagebox
import numpy as np

def calculate_bayesian_probs():
  # Initial coin probabilities (probability of Head)
  p = np.array([0.12, 0.27, 0.21, 0.96])

  # Initial prior: all coins are equally likely
  posterior = np.ones(4) / 4

  # Read and preprocess user input
  sequence = entry.get().upper().replace(" ", "")
  if not sequence or not all(c in 'HT' for c in sequence):
    messagebox.showerror("Input Error", "Please enter a sequence containing only 'H' or 'T'")
    return

  # Clear previous results from the table
  for row in tree.get_children():
    tree.delete(row)

  # Bayesian updating for each observation
  for i, o in enumerate(sequence, 1):
    likelihood = p if o == 'H' else (1-p)
    posterior = posterior * likelihood
    posterior /= posterior.sum()

    # Probability of Head in the next flip
    next_p_h = round((posterior * p).sum(), 2)

    # Insert result into the table
    tree.insert("", "end", value=(i, o, f"{next_p_h:.2f}"))

# GUI Setup
root = tk.Tk()
root.title("Bayesian Coin Flip Tracker")
root.geometry("400x450")

# Top input frame
frame_top = tk.Frame(root)
frame_top.pack(fill="x", padx=10, pady=10)

tk.Label(frame_top, text="Enter flip results (H - Head, T - Tail):").pack(pady=5)

entry = tk.Entry(frame_top)
```

```python
entry.pack(pady=5, fill="x")
entry.insert(0, "HHHTHTHHH") # Default example

btn_calc = tk.Button(frame_top, text="Calculate", command=calculate_bayesian_probs, bg="#4CAF50", fg="white")
btn_calc.pack(pady=10, fill="x")

# Results table
columns = ("test_no", "observed", "next_p")
tree = ttk.Treeview(root, columns=columns, show="headings")

tree.heading("test_no", text="Test #")
tree.heading("observed", text="Observed result")
tree.heading("next_p", text="P(H) in next flip")

tree.column("test_no", width=80, anchor="center")
tree.column("observed", width=120, anchor="center")
tree.column("next_p", width=150, anchor="center")

tree.pack(pady=10, padx=10, fill="both", expand=True)

root.mainloop()
```